Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №4

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Листинг: 6
1.c 6
2.c 6
Main.c 6

Задание: разработайте приложение, запускающее несколько программ. Определите идентификаторы соответствующих процессов. Установите родственные связи между ними.

Цель: получение навыков использования функций API создания процессов на платформе Linux

Ход работы:

Создадим программы, которые в будущем будут выполнять роль дочерних (листинг 1.c 2.c). После запуска программы в терминале будет выводиться сообщение для распознавания программы.

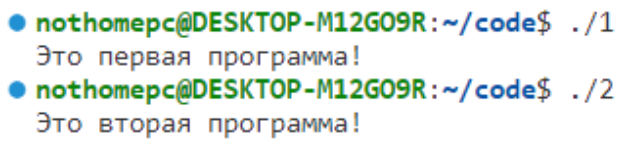


Рисунок 1 – Вывод программы

Создадим родительскую программу **main** (листинг Main.c). Она выполняется в главном процессе (родительском процессе) и создает два дочерних процесса. В этой программе выполняются следующие шаги:

1. Создание первого дочернего процесса (child_pid1) с использованием функции **fork()**.
2. В родительском процессе (main) выводится сообщение о том, что это родительский процесс, и печатается его идентификатор процесса (PID).
3. Внутри первого дочернего процесса (child_pid1 == 0) также выводится сообщение о том, что это дочерний процесс, и печатается его идентификатор процесса (PID). Затем с помощью **execl()** запускается первая программа "./1" с указанным именем "первая_программа". Если запуск программы завершается успешно, процесс завершается с кодом EXIT_SUCCESS, иначе с кодом EXIT_FAILURE.
4. После создания и запуска первого дочернего процесса повторяются аналогичные шаги для создания и запуска второго дочернего процесса (child_pid2) и второй программы "./2" с именем "вторая_программа".
5. После создания и запуска обоих дочерних процессов в родительском процессе используется функция **wait(NULL)** для ожидания завершения обоих дочерних процессов. Это гарантирует, что родительский процесс не завершится, пока оба дочерних процесса не завершат свою работу.

6. После завершения обоих дочерних процессов родительская функция выводит сообщение "Родительский процесс завершил выполнение" и завершает свою работу.

```
nothomepc@DESKTOP-M12GO9R:~/code$ gcc -o app -Wall main.c
nothomepc@DESKTOP-M12GO9R:~/code$ ./app
Это родительский процесс (PID 8562)
Это первый дочерний процесс (PID 8563)
Это родительский процесс (PID 8562)
Это второй дочерний процесс (PID 8564)
Это первая программа!
Это вторая программа!
Родительский процесс завершил выполнение
```

Рисунок 2 – Компиляция, запуск и результат работы программы.

Вывод:

Нами было разработано приложение, запускающее несколько программ; определены идентификаторы соответствующих процессов; установлены родственные связи между ними.

Так же нами были получены навыки использования функций API создания процессов на платформе Linux

Листинг:

1.c

```c
#include <stdio.h>

int main() {
    printf("Это первая программа!\n");
    return 0;
}
```

2.c

```c
#include <stdio.h>

int main() {
    printf("Это вторая программа!\n");
    return 0;
}
```

Main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t child_pid1, child_pid2;

    child_pid1 = fork();

    if (child_pid1 < 0) {
        perror("Ошибка при создании первого дочернего процесса");
        exit(EXIT_FAILURE);
    } else if (child_pid1 == 0) {
        printf("Это первый дочерний процесс (PID %d)\n", getpid());
        execl("./1", "первая_программа", NULL);
        perror("Ошибка при запуске первой программы");
        exit(EXIT_FAILURE);
    } else {
        printf("Это родительский процесс (PID %d)\n", getpid());
    }

    child_pid2 = fork();

    if (child_pid2 < 0) {
        perror("Ошибка при создании второго дочернего процесса");
        exit(EXIT_FAILURE);
    } else if (child_pid2 == 0) {
        printf("Это второй дочерний процесс (PID %d)\n", getpid());
        execl("./2", "вторая_программа", NULL);
```

```
        perror("Ошибка при запуске второй программы");
        exit(EXIT_FAILURE);
    } else {
        printf("Это родительский процесс (PID %d)\n", getpid());
    }

    wait(NULL);
    wait(NULL);

    printf("Родительский процесс завершил выполнение\n");

    return 0;
}
```